MODULE II

DC MOTORS AND THEIR INTERFACING WITH ARDUINO

Using Transistors and Driving DC Motors: Driving DC Motors with Transistor, Protection Diodes, Controlling Motor Speed with PWM, H-Bridge to Control DC Motor Direction, Roving Robot, Powering, Geared Motor with 7805 Regulator, Robot Hardware and Software Driving Stepper and Servo Motors: Driving Servo Motors, controlling a Servo, Driving Stepper Motors, Bipolar Stepper Motors Working

Books: Exploring Arduino Jeremy Blum, Arduino Cookbook Michael Margolis et. Al

DC MOTORS AND THEIR INTERFACING WITH ARDUINO

DC motors are widely used electromechanical devices that rotate continuously when a DC voltage is applied across them. They are commonly found in household appliances, RC cars, power tools, DVD players, and robotic applications due to their low cost, wide size range, and simple control.

The speed of a DC motor can be controlled by varying the applied voltage, while gearboxes are used to trade speed for increased torque. The direction of rotation can be reversed by changing the polarity of the supply voltage, typically achieved using an H-bridge circuit.

Brushed DC motors consist of stationary magnets (stator) and a rotating coil (rotor). Electrical contact is made using carbon brushes, which periodically reverse the current direction in the coil, producing continuous rotation due to magnetic attraction and repulsion. Although brushed motors are inexpensive and easy to control, their lifespan is limited because the brushes wear out over time.

DC motors are inductive loads and require higher current than what an Arduino I/O pin can supply. They also generate high voltage spikes when switched OFF.

To safely interface a DC motor with an Arduino:

- A transistor is used as a switch or amplifier to handle high current.
- A flyback (protection) diode is connected across the motor to prevent reverse voltage damage.
- A separate power supply is often used for the motor.

- PWM (Pulse Width Modulation) is used to control motor speed.

Thus, proper isolation and protection are essential when driving DC motors using microcontrollers.

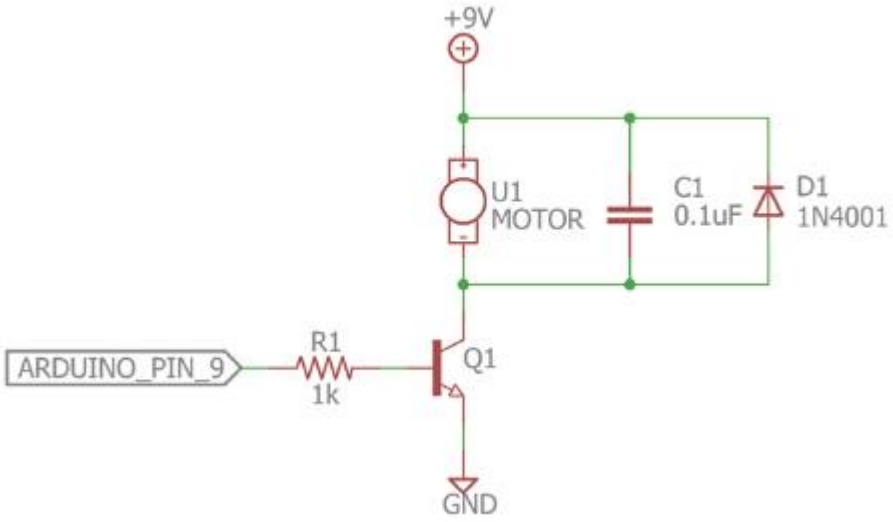


Figure 2-1: DC motor control schematic

- Q1 is an NPN bipolar junction transistor (BJT) used for switching the separate 9V supply to the motor.
- A 1kΩ resistor is used to separate the transistor's base pin from the control pin of the Arduino. It limits the current that flows to and from the gate pin.
- U1 is the DC motor.
- C1 is for filtering noise caused by the motor.
- D1 is a diode used to protect the power supply from reverse voltage caused by the motor acting like an inductor. This is commonly called a flyback, snubber, or freewheeling diode

A transistor can be used as an electrically controlled switch to safely drive a DC motor using a microcontroller like Arduino. A Bipolar Junction Transistor (BJT) has three terminals: Emitter (E), Base (B), and Collector (C). When a sufficient voltage is applied to the base, the transistor operates in saturation mode, allowing current to flow freely from collector to emitter, similar to a closed switch. This enables the motor to run.

The Arduino I/O pin (5V) provides the base bias required to turn ON the transistor. By applying PWM (Pulse Width Modulation) at the base, the transistor is rapidly switched ON and OFF, thereby controlling the speed of the DC motor. The motor's inertia smooths the switching, and the PWM duty cycle determines the motor speed.

Since DC motors are inductive loads, they generate high reverse voltage spikes when power is suddenly removed. To protect the transistor and power supply, a flyback (protection) diode is connected across the motor. This diode provides a safe path for the induced current and prevents harmful reverse voltages.

A secondary power supply (such as a 9V battery or adapter) is used to power the motor because:

- DC motors draw more current than the Arduino 5V supply can provide.
- Many motors require voltages higher than 5V.

The grounds of the Arduino and the external supply must be connected to establish a common reference.

CONTROLLING MOTOR SPEED WITH PWM

The speed of a DC motor can be effectively controlled using Pulse Width Modulation (PWM), similar to controlling LED brightness in Arduino applications. A PWM-capable output pin of the Arduino sends a signal with a varying duty cycle to the base of a transistor that drives the motor.

By rapidly switching the transistor ON and OFF, the current through the motor is pulsed. Due to the motor's inertia and inductive nature, these rapid changes are averaged out, resulting in a smooth change in motor speed.

- Higher duty cycle → Motor ON for longer time → Higher speed
- Lower duty cycle → Motor ON for shorter time → Lower speed

```
//Simple Motor Speed Control Program

const int MOTOR=9; //Motor on Digital Pin 9
void setup( )
{
pinMode (MOTOR, OUTPUT);
}
void loop()
{
for (int i=0; i<256; i++)
{
analogWrite(MOTOR, i);
delay(10);
}
```

```
delay(2000);
for (int i=255; i>=0; i--)
{
analogWrite(MOTOR, i);
delay(10);
}
delay(2000);
}
```

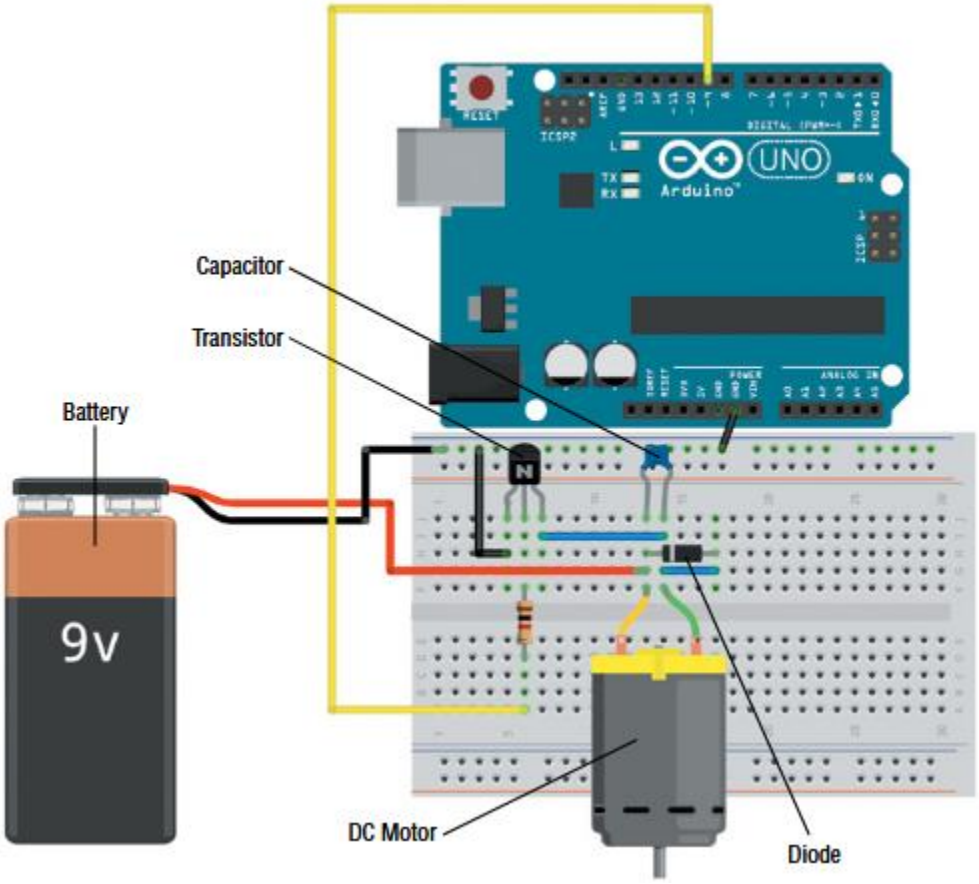
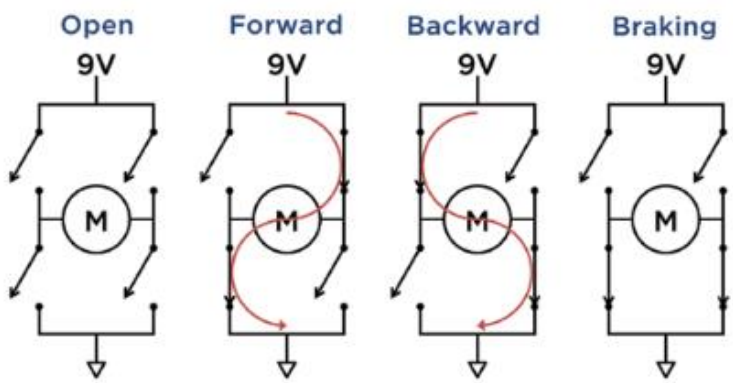


Figure 2-2: DC motor wiring

USING AN H-BRIDGE TO CONTROL DC MOTOR DIRECTION

Any useful DC motor needs to be able to spin in two directions. To accomplish this, you can use a handy device called an H-bridge.

The operation of an H-bridge

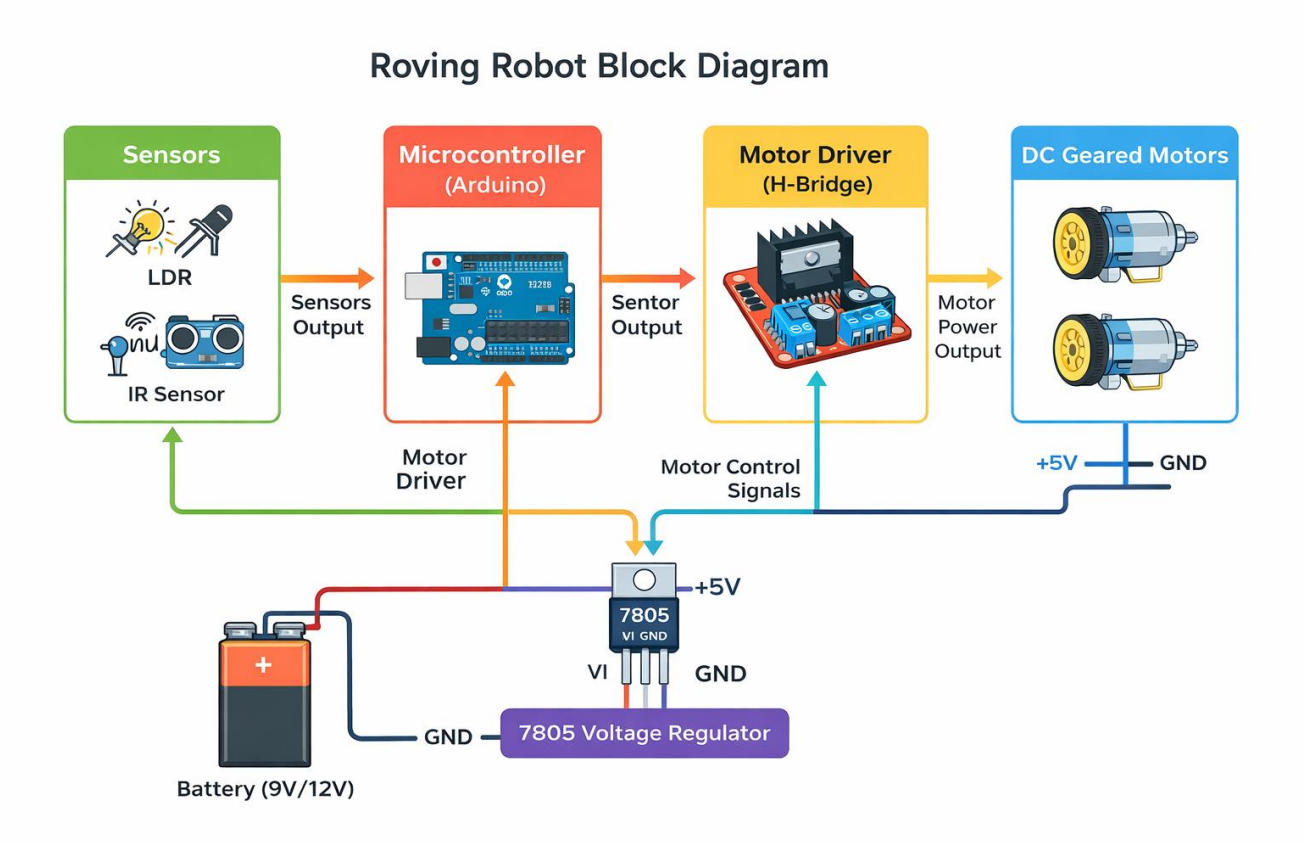


The motor in combination with the four switches forms an uppercase H. Although the diagram shows them as switches, the switching components are actually transistors. The H-bridge has four main states of operation: open, forward, backward, and brak- ing. In the open state, all the switches are open and the motor doesn't spin. In the forward state, two diagonally opposing switches are engaged, causing current to flow from 9V, through the motor, and down to ground. When the opposing switches are flipped, current then runs through the motor in the opposite direction, causing it to spin backward. If the H-bridge is put in the braking state, all residual motion caused by momentum is ceased, and the motor stops.

ROVING ROBOT

A roving robot is a mobile robotic system that can move freely in an environment while sensing external conditions and responding intelligently. Unlike stationary robots, a roving robot is capable of navigation and decision-making based on sensor inputs. It combines mechanical motion with electronic control and embedded software, making it an important application of robotics and embedded systems.

The operation of a roving robot is based on the sense–think–act principle. Sensors act as the sensing unit and detect environmental parameters such as light, distance, or obstacles. The controller processes this sensor data and decides the appropriate action. Finally, actuators such as DC motors perform physical movement based on the controller's decision. This continuous cycle enables autonomous movement.



The block diagram of a roving robot consists of sensors, a microcontroller, a motor driver, DC motors, and a power supply. Sensors provide input to the microcontroller, which processes the information and sends control signals to the motor driver. The motor driver then operates the motors, allowing the robot to move forward, backward, or turn. The power supply provides the required electrical energy to all components.

Sensors play a vital role in the functioning of a roving robot. Commonly used sensors include LDRs for light detection, IR sensors for obstacle or line detection, and ultrasonic sensors for distance measurement. These sensors convert physical quantities into electrical signals that can be processed by the microcontroller.

The microcontroller, commonly an Arduino board, acts as the brain of the roving robot. It reads the sensor values through its input pins and executes a predefined program. Based on the logic written in the program, it generates control signals for the motor driver. The flexibility of the microcontroller allows easy modification of robot behavior through software changes.

Since DC motors require higher current than a microcontroller can supply, a motor driver circuit using an H-bridge is employed. The H-bridge allows the motor to rotate in both forward and

reverse directions and also enables speed control using Pulse Width Modulation (PWM). Motor driver ICs such as L293D or L298 are commonly used in roving robots.

The robot uses DC geared motors for movement. Geared motors are preferred because they provide high torque at low speeds, which is ideal for robotic applications. By controlling the speed and direction of the left and right motors independently, the robot can move straight, turn left, turn right, or stop.

Powering the motors directly from the Arduino is not recommended because motors draw large current, especially during startup and stall conditions. To overcome this, a separate regulated power supply is used for the motors. This prevents voltage drops and protects the controller from unexpected resets or damage.

A 7805 voltage regulator is commonly used to power the geared DC motors. The 7805 is a linear voltage regulator that provides a fixed 5V output from a higher input voltage such as a 9V battery. It has three pins: input, ground, and output. Capacitors are connected at the input and output terminals to reduce voltage ripples and improve stability.

In a roving robot, the hardware components include the Arduino board, sensors, motor driver IC, DC geared motors, 7805 voltage regulator, battery, chassis, and connecting wires. The chassis provides mechanical support and holds all components in position. Proper hardware integration ensures reliable robot operation.

The robot software is written using the Arduino IDE in Embedded C or C++. The software defines how sensor data is read, how decisions are made, and how motors are controlled. The program typically consists of an initialization section to configure pins and a loop section that continuously reads sensors and controls motor movement.

The working of a roving robot can be understood through an example of a light-following robot. When light falls more on the left sensor, the controller increases the speed of the right motor, causing the robot to turn toward the light source. When both sensors receive equal light, the robot moves forward. This demonstrates autonomous behavior based on sensor input.

Roving robots are widely used in applications such as space exploration, military surveillance, search and rescue operations, industrial inspection, and educational robotics. Their ability to operate in hazardous or inaccessible environments makes them highly valuable in real-world scenarios.

The advantages of roving robots include reduced human effort, the ability to work in dangerous environments, flexibility in programming, and cost-effective implementation for learning and research. However, they also have limitations such as limited battery life, dependency on sensors, and restricted intelligence.

DRIVING STEPPER AND SERVO MOTORS

STEPPER MOTOR CONTROL

A stepper motor is a brushless DC motor that moves in discrete steps rather than rotating continuously. Each electrical pulse applied to the motor causes it to rotate by a fixed angular displacement. Because of this step-by-step motion, stepper motors provide high positional accuracy and repeatability, making them suitable for robotics, CNC machines, and automation systems.

Bipolar Stepper Motors:

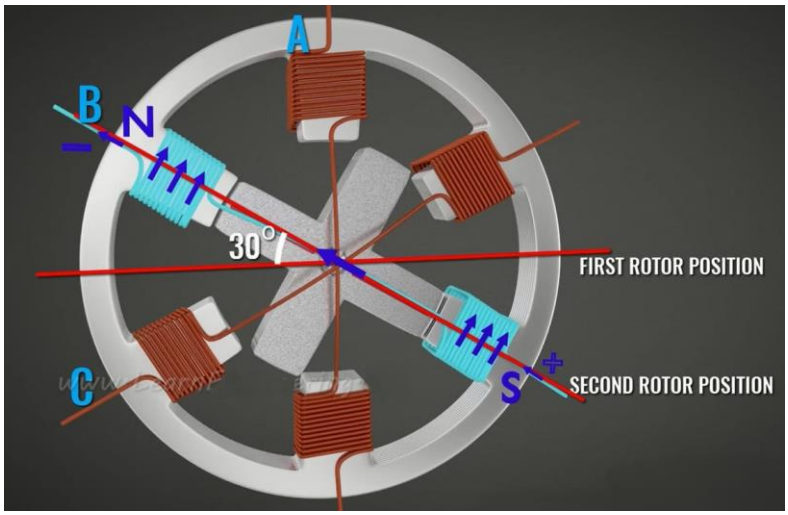
Bipolar stepper motors use two phases, and current flows through each phase in both directions. This bidirectional current flow allows bipolar stepper motors to produce higher torque compared to unipolar motors. However, this requires the use of an H-bridge driver circuit.

Construction of a Bipolar Stepper Motor

A bipolar stepper motor consists of a **permanent magnet rotor** mounted on a shaft and multiple **electromagnetic coils** arranged around it. The coils are wound around soft iron cores to form electromagnets. These coils are grouped into two phases, and the coils of each phase are placed alternately around the rotor. The NEMA-17 motor achieves **200 steps per revolution** by using multiple coils and a rotor with several alternating north and south poles.

Working Principle of Bipolar Stepper Motor

The working of a bipolar stepper motor is based on the **sequential energization of coils**. When current flows through a coil, it generates a magnetic field. The permanent magnet rotor aligns itself with this magnetic field. By switching the current ON and OFF in different coils and reversing the current direction, a rotating magnetic field is produced, causing the rotor to move step by step.



$$Step\ angle = \frac{No\ of\ Stator - Number\ of\ Rotor}{No\ of\ Stator \times Number\ of\ Rotor} \times 360$$

Step-by-Step Operation

Initially, when no current flows through any coil, no magnetic field is produced and the rotor can rotate freely. When **Phase-1** is energized, a magnetic field is created that locks the rotor in position. Next, Phase-1 is turned OFF and **Phase-2** is energized, shifting the magnetic field by 90 degrees and causing the rotor to rotate. The current direction in Phase-1 is then reversed using an H-bridge, rotating the rotor further. The same process is repeated for Phase-2 in the opposite direction. Continuous repetition of this sequence produces smooth step-by-step rotation.

Driving a Stepper Motor Using an H-Bridge

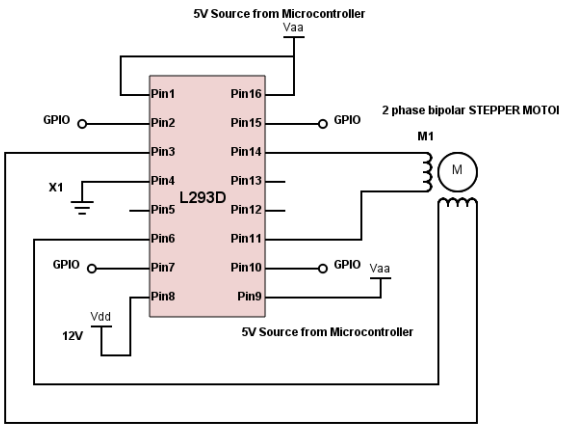
A bipolar stepper motor is electrically similar to **two DC motors** that must be driven bidirectionally. Each phase of the stepper motor requires one H-bridge. An IC such as **L293D** is commonly used. Unlike DC motor control, stepper motors require **precise timing and sequencing** of the coils to maintain accurate motion.

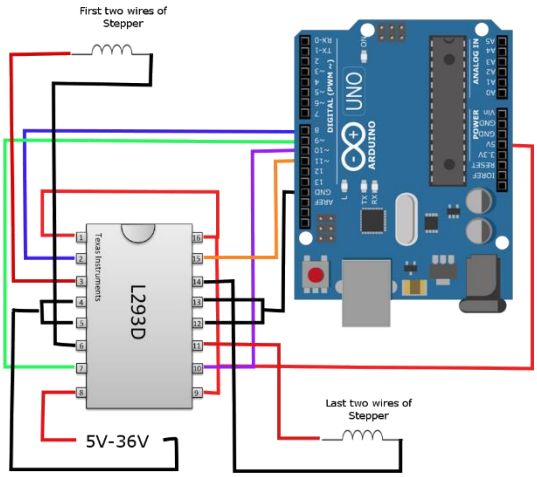
Stepper Motor Power Requirements

Stepper motors consume **high current** and are typically driven at **12V**. Supplying this power directly from batteries is inefficient. Therefore, the PDF recommends using a **12V DC wall adapter** connected to the Arduino barrel jack. The Arduino's onboard regulator supplies 5V for logic circuits, while the VIN pin provides 12V directly to the motor driver.

Stepper Motor Wiring

Correct wiring is essential for stepper motor operation. Each coil pair must be identified and connected to the correct H-bridge outputs. A multimeter can be used to identify coil pairs by checking resistance. Reversing a coil pair changes the direction of rotation but does not damage the motor.





Controlling Stepper Motor Using Arduino

The Arduino **Stepper library** simplifies control by automatically handling coil excitation sequences. The programmer only needs to specify:

- Number of steps per revolution (typically 200)
- Speed in RPM
- Direction and number of steps

The step( ) function moves the motor by a specified number of steps and is a **blocking function**, meaning the program waits until motion is completed.

Advantages of Stepper Motor Control

Stepper motors provide **precise position control**, high torque at low speeds, and reliable open-loop operation without feedback sensors. Their predictable behavior makes them ideal for precision motion systems.

Applications

Stepper motors are widely used in:

- Robotics
- CNC machines
- 3D and 2D printers

- Industrial automation
- Instrument positioning systems